MSPM0G3507/MSPM0G3519 竞赛小车教程 — 钢珠控制 + 八路巡线综合

MSPM0G3507/MSPM0G3519 竞赛小车教程 — 钢珠控制 + 八路巡线综合

- 一、硬件平台
- 二、软件环境
- 三、结构说明
  - 3.1 组件清单与物料准备
  - 3.2实物外观图
  - 3.3组装说明
- 四、简要原理
  - 4.1 接线引脚
  - 4.2 实物接线图
  - 4.3 控制原理
- 五、工程文件结构
- 六、核心参数配置
  - 6.1 舵机控球 PID (app_k230.c)
  - 6.2 巡线 PID (app_irtracking.c)
  - 6.3 各任务巡线速度 (questions.c)
  - 6.4 任务超时设置 (questions.c)
- 七、任务说明
  - 7.1 状态机结构 (questions.h)
  - 7.2 任务详细说明
- 八、关键代码讲解
  - 8.1 K230 控球 — K230_Servo_Control() (app_k230.c)
  - 8.2 巡线 — LineWalking() (app_irtracking.c)
  - 8.3 巡线+控球组合 — LineWalking_WithBall() (app_irtracking.c)
  - 8.4 任务2 钢珠往复 (questions.c)
- 九、操作流程

一、硬件平台

组件	型号/说明
主控	MSPM0G3507/MSPM0G3519 开发板
摄像头	K230 AI 视觉模块 (YOLOv8 钢珠检测)
巡线	八路红循迹模块 (I2C 0x12)
显示	OLED 128×32 (I2C)
舵机	9G舵机 (钢珠倾斜控制)
电机	两轮L型520编码电机 (串口驱动板)
按键	扩展板K1单键控制

二、软件环境

- **IDE:** Keil MDK V5 (ARM Compiler V6.7)
- **SDK:** TI MSPM0 SDK 2.02.00.05

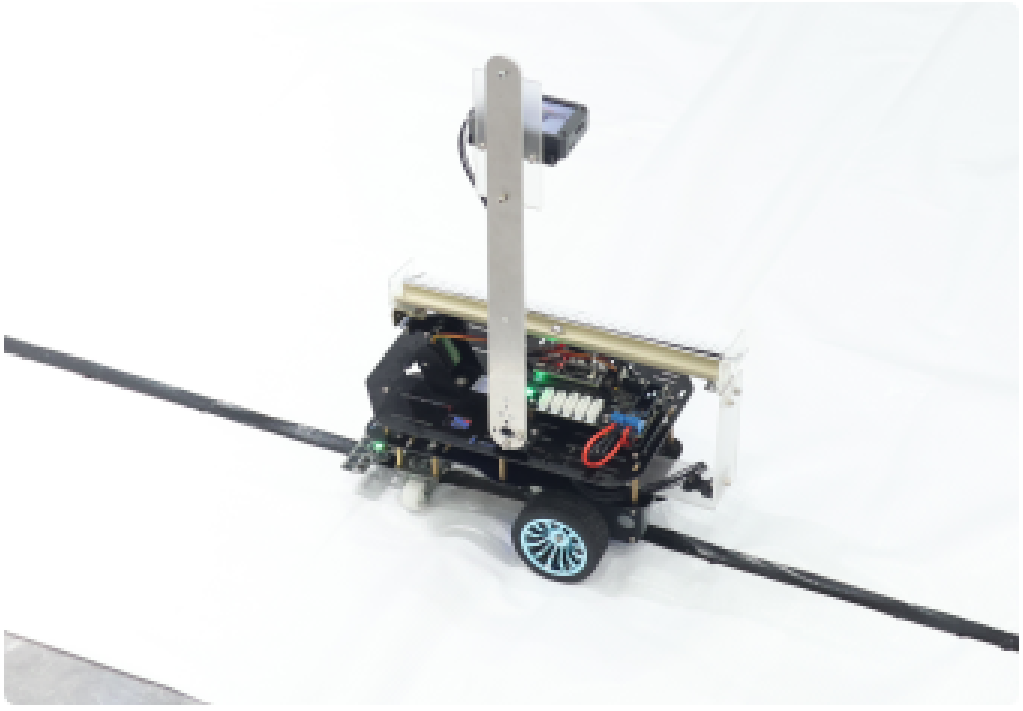
三、结构说明

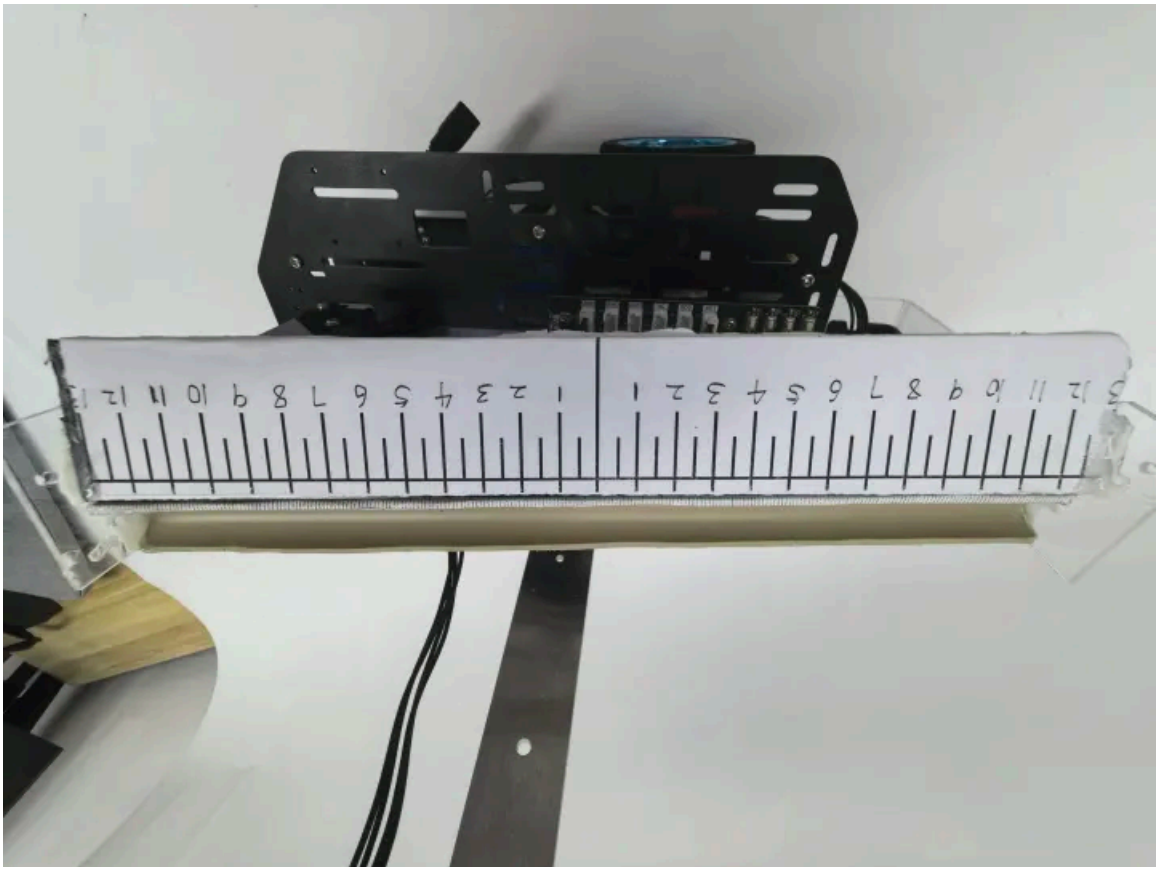
3.1 组件清单与物料准备

- **底盘部分**：黑色金属蚀刻底板、驱动电机、车轮。底盘上带有预留的螺丝孔和走线孔。
- **机械臂结构**：蓝色/黑色金属舵机支架、金属连杆、金属L型/一字型连接件、舵机、金属法兰盘。
- **水平标尺滑台**：透明亚克力外壳、手写数字标尺及零位参考线、内部传动导轨/丝杆。
- **顶部视觉支架**：拉丝金属长条（铝合金/不锈钢）、透明亚克力L型支架、K230模块。
- **五金配件**：M2/M3 组合螺丝、螺母、垫片、黑色扎带、热缩管。

注意：除了v2底盘套装里面的配件，9g舵机，K230以及循迹模块等可以在我们店铺购买。其他的连接杆，标尺，水管以及那些连接五金配件需要自己想办法获取

3.2实物外观图



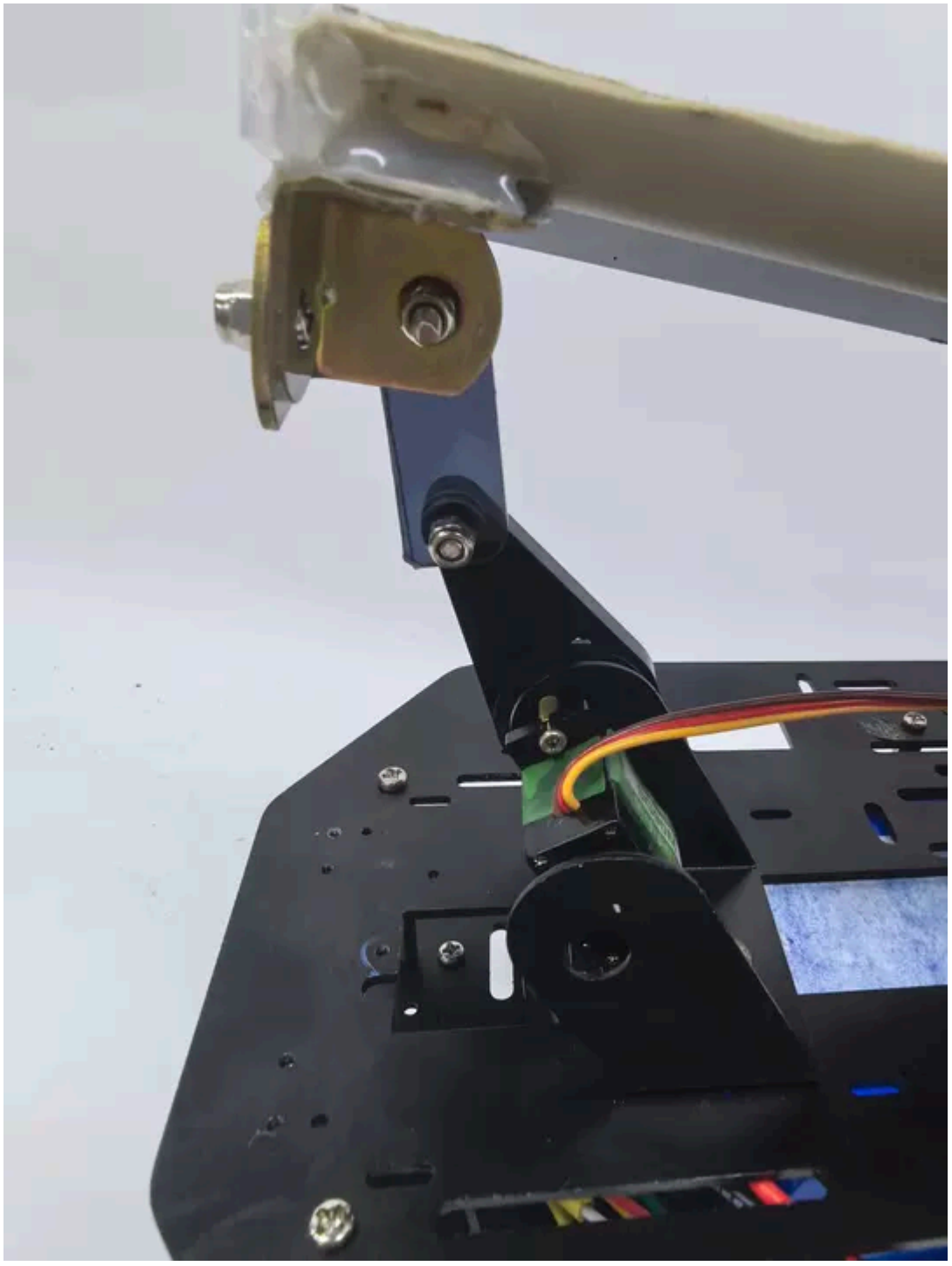



SG90M









3.3组装说明

步骤 1：安装移动底盘与电机

- 将两个驱动电机（带车轮）分别固定在黑色金属底板的两侧。确保车轮朝向一致，且与地面充分接触。
- 安装主控板mspm0g3507，使用螺丝固定在底盘预留位。注意主控板避开车轮和电机的转动空间。

步骤 2：搭建水平标尺与平移滑台

- *参考图5*：将透明亚克力外壳组装好，并将带有刻度标尺的纸板嵌入内部。这个结构固定在底盘的前方。
- 注意滑台内部的导轨或丝杆需要保持水平，这是后续机械臂坐标校准的关键参考基准。

步骤 3：组装机械臂关节

- *参考图3、图7*：将绿色舵机卡入黑色金属支架中，用配套螺丝锁紧。
- 安装舵机臂，连接蓝色/黑色的金属连杆。
- 参考图7搭建末端执行器（连接件/爪部），图片中该部件使用了较多**透明亚克力与热熔胶/树脂**加固。实际组装中建议使用3D打印件或金属件代替，以保证足够的结构强度。
- 将舵机底座固定在底盘后方（参照图7的布局位置）。

步骤 4：安装垂直金属支架

- 参考图1、图4、图6：取拉丝金属长条，将其垂直安装到底盘中央或侧前方的固定槽位上。
- 使用螺丝固定时，确保金属条垂直于底盘平面，不能摇晃。图6中的螺母是从背部贯穿锁紧的。

步骤 5：安装顶部视觉传感器

- 参考图35：将K230模块的背面朝外。使用透明亚克力L型支架和M3螺丝，将模块固定在垂直金属支架的顶端。
- 注意调整传感器的朝向，确保它能够看到前方的轨道和标尺。

四、简要原理

4.1 接线引脚

四路电机驱动板:

电机驱动板	MSPM0G3507/MSPM0G3519
RX2	PB6
TX2	PB7
GND	GND
5V	5V

电机接线:

电机	驱动板
M2	M-
V	3V3
A	H1B
B	H1A
G	GND
M1	M+

八路巡线模块:

巡线模块	MSPM0G3507/MSPM0G3519
5V	5V
SCL	PA15
SDA	PA16
GND	GND

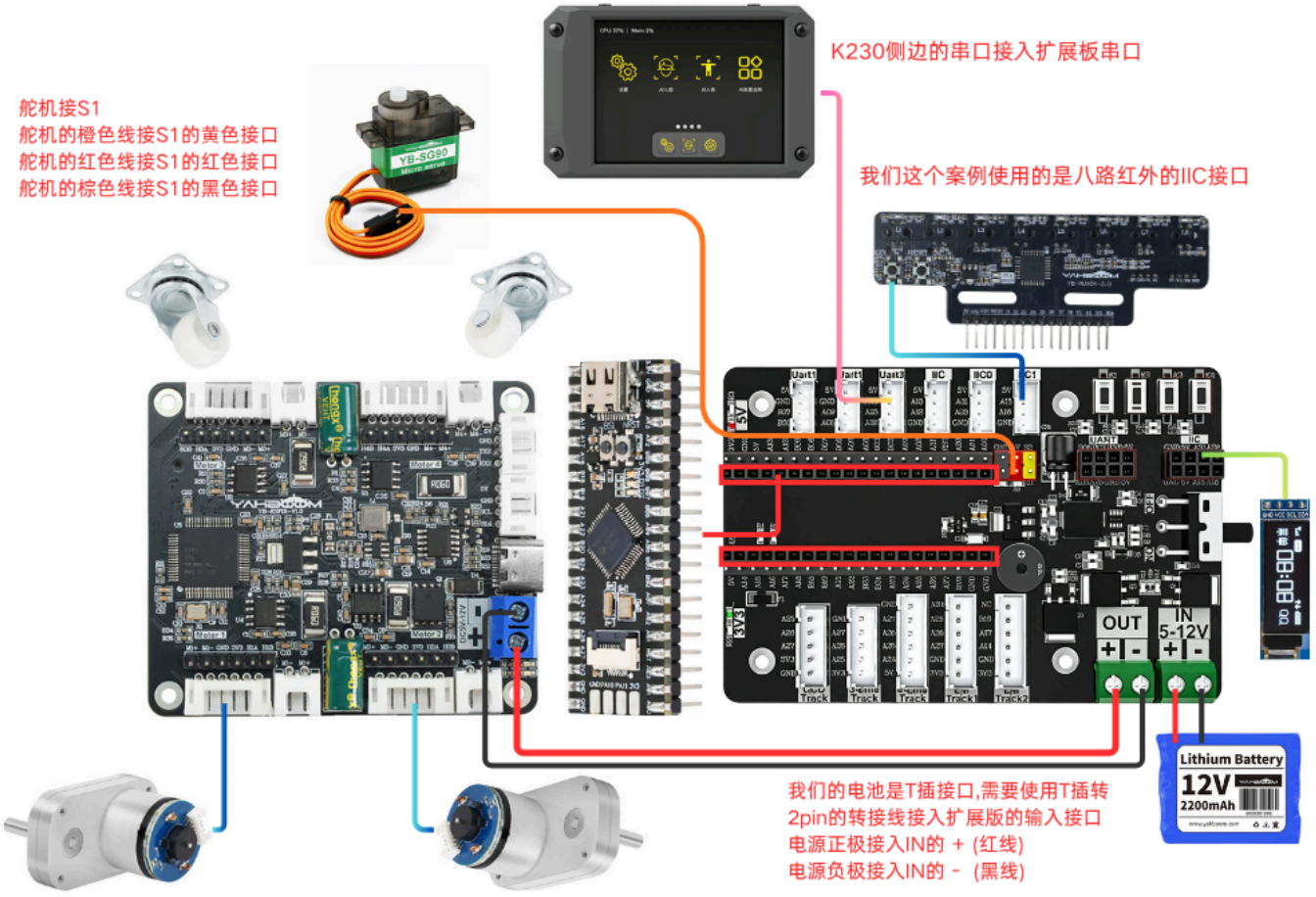
OLED:

OLED	MSPM0G3507/MSPM0G3519
VCC	5V
GND	GND
SDA	PA28
SCL	PA31

K230 摄像头:

K230	MSPM0G3507/MSPM0G3519
5V	5V
GND	GND
TXD	PA3
RXD	PA2

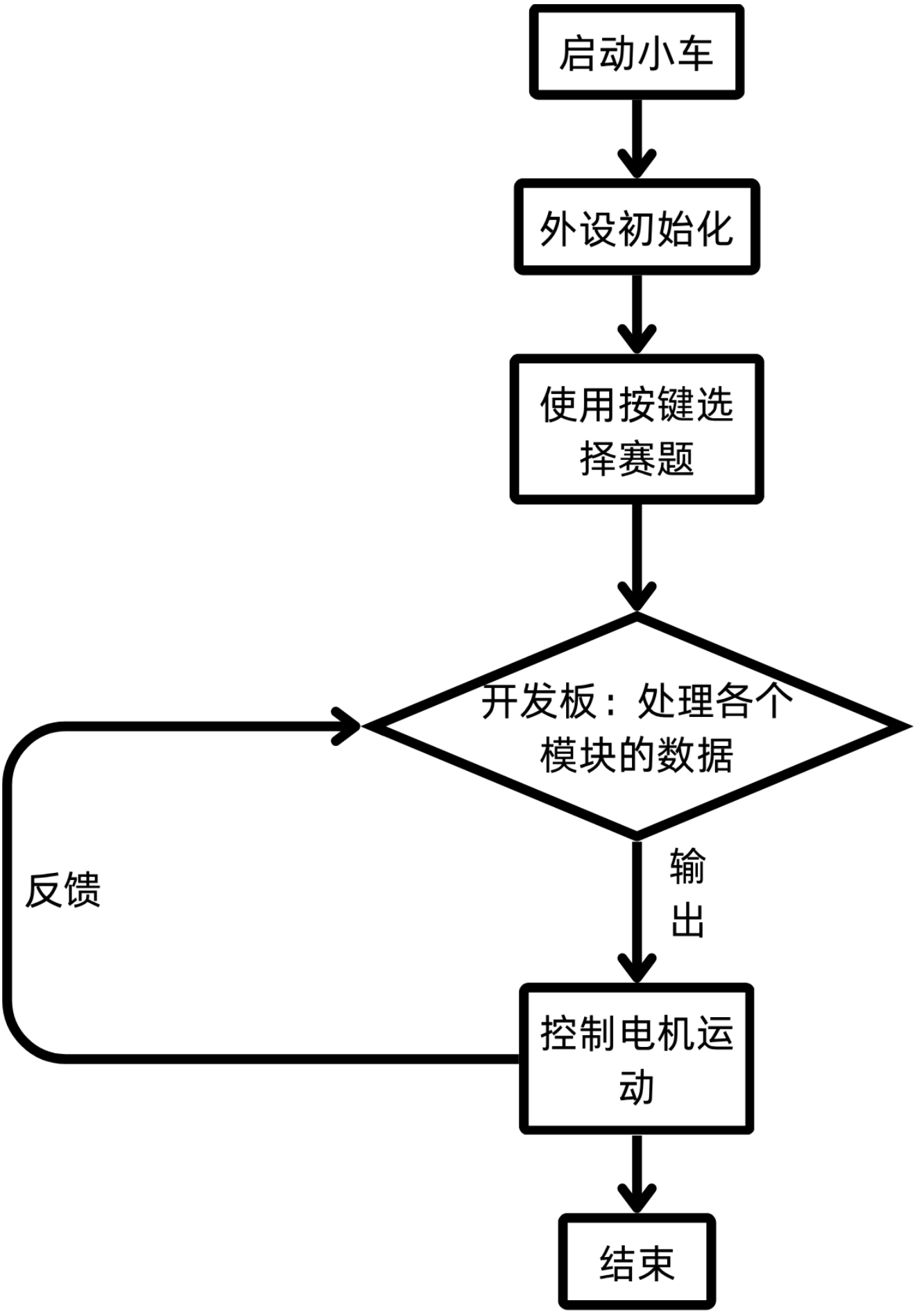
4.2 实物接线图



整体结构采用的是2路编码电机搭配2路万向轮的结构控制小车进行移动的

4.3 控制原理

小车巡线控制流程:



1. 八路红外传感器读取黑线位置 -> 计算偏差值
2. PID 控制器计算转弯修正量 -> 控制电机差速转向

3. K230 摄像头检测钢珠位置 -> 舵机 PID 控制摆杆角度
4. 定时器超时或途经点检测 -> 停车/切换状态

PID 控制参数说明:

PID参数	作用	调节方向
P (比例)	决定响应速度, 偏差大时输出大	太大: 振荡蛇形; 太小: 转弯无力
I (积分)	消除稳态偏差, 累积误差修正	太大: 过冲; 太小: 偏一边回不来
D (微分)	抑制振荡, 提前减速	太大: 抖动; 太小: 刹不住

五、工程文件结构

```
Competition_PJ/
├── main.c                      # 主程序: 外设初始化, 主循环, 状态机分发
├── APP/
|   ├── app_k230.c/h            # K230 串口协议 + PID 舵机控球 (核心)
|   ├── app_pid.c/h             # PID 控制器 (位置式 + 增量式)
|   ├── app_kalman.c/h          # 一维卡尔曼滤波器
|   └── app_servo.c/h           # 舵机 PWM 驱动
├── BSP/
|   ├── Eight_Tracking/
|   |   └── app_irtracking.c/h # 八路巡线 + 巡线PID + 组合任务(巡线+控球)
|   ├── Motor/
|   |   ├── app_motor.c/h       # 小车运动控制 (Motion_Car_Control)
|   |   └── app_motor_usart.c/h# 电机驱动板串口协议
|   ├── OLED/
|   |   ├── oled.c/h            # OLED 驱动 + 显示函数 OLED_time()
|   |   └── oledfont.h          # 字体数据
|   ├── Key/
|   |   └── key.c/h             # 按键驱动 + 模式选择状态机
|   ├── Questions/
|   |   └── questions.c/h       # 竞赛任务状态机 (Task 1~5)
|   ├── Task/
|   |   └── task.c/h            # 定时任务调度器 (Scheduler_Run)
|   ├── Timer/
|   |   └── timer.c/h           # 1ms 系统时钟 Get_Time()
|   └── usart.c/h               # 串口初始化 (UART0 debug, UART1 电机, UART3 K230)
└── ti_msp_dl_config.c/h        # TI 驱动库配置 (SysConfig 生成)
```

六、核心参数配置

6.1 舵机控球 PID (app_k230.c)

```
#define PID_KP          0.08f   // 比例系数: 响应力度, 大则猛推
#define PID_KI          0.003f  // 积分系数: 消除稳态偏移
#define PID_KD          2.86f   // 微分系数: 主要阻尼, 抑制振荡
```

```
#define K230_CENTER    360.0f  // K230 图像中心 X 像素
#define K230_MIN        99.0f  // 左边界像素
#define K230_MAX       599.0f  // 右边界像素
#define FOV_CM          28.0f  // 摄像头视野宽度 (cm)
#define PIPE_HALF_CM    12.5f  // 水管半长 (cm), 水管全长 25cm

#define DEADBAND_PX      3.0f  // 死区 (px), 误差<3px 不动作
#define RATE_LIMIT       6.0f  // 单帧 CC 变化上限, 防舵机猛转
#define KALMAN_Q         1.0f  // 卡尔曼过程噪声 (大=响应快)
#define KALMAN_R         0.3f  // 卡尔曼测量噪声 (大=平滑)
```

双向 px_per_cm 补偿 (摄像头不完全居中):

```
目标 >= 0 (右侧): px_per_cm = (599-360)/12.5 = 19.12
目标 <  0 (左侧): px_per_cm = (360-99)/12.5  = 20.88
```

PID 调参指南:

现象	调哪个	方向	说明
球在中心附近来回晃	↑ **KD**	加大阻尼	KD 越大刹车越早
球推不到目标就停了	↑ **KP**	加大推力	KP 越大推越远
球冲过目标弹很远	↓ **KP**	减小推力	
球稳定后偏一边不回中	↑ **KI**	加强积分	积分越大归中越快
球不动时舵机抖动	↑ **DEADBAND**	放宽死区	

6.2 巡线 PID (app_irtracking.c)

```
#define IRTrack_Trun_KP   (150)   // 巡线 P: 越大纠偏越猛
#define IRTrack_Trun_KI   (0.15)  // 巡线 I: 消除稳态偏差
#define IRTrack_Trun_KD   (1.5)   // 巡线 D: 抑制振荡
#define IRR_SPEED         250     // 巡线默认速度 (mm/s)
```

巡线调参指南:

现象	调哪个	方向
转弯不够/压线	↑ **KP**	加大
蛇形走位	↑ **KD** 或 ↓ **KP**	阻尼/减推
一直偏一边	↑ **KI**	加强积分

惯性补偿 (巡线+控球组合任务用):

```
#define BALL_INERTIA_FWD   -0.8f  // 前进惯性补偿 (cm), 值越大球越前推
#define BALL_INERTIA_TURN  -0.8f  // 转弯惯性补偿 (cm)
#define BALL_START_COMP    -5.0f  // 起步补偿 (cm), 车加速球滞后时前推
#define BALL_STOP_COMP      5.0f  // 停车补偿 (cm), 车停球前冲时回拉
#define RAMP_TIME_MS       3000   // 起步缓加速时间 (ms), 越大起步越缓
```

6.3 各任务巡线速度 (questions.c)

```
#define TASK1_SPEED      316   // 任务1 纯循迹速度
#define TASK3_SPEED      130   // 任务3 A->B+控球速度
#define TASK4_SPEED      222   // 任务4 一圈+控球速度
#define TASK5_SPEED      222   // 任务5 指定位置速度
#define RAMP_TIME_MS     3000  // 起步缓加速时间
```

6.4 任务超时设置 (questions.c)

```
#define TASK1_MAX_MS   20000    // 任务1 纯循迹 20s 超时
#define TASK2_MAX_MS    5000    // 任务2 钢珠往复 5s 超时
#define TASK3_MAX_MS    8000    // 任务3 A->B+控球 8s 超时
#define TASK4_MAX_MS   30000    // 任务4 一圈+控球 30s 超时
#define TASK5_MAX_MS   30000    // 任务5 指定位置 30s 超时
```

七、任务说明

7.1 状态机结构 (questions.h)

主状态 (Main_State):

宏	值	任务描述
STOP_STATE	0	停止
TASK_LINE_LAP	1	任务1: 纯循迹一圈回A, ≤20s
TASK_BALL_CTRL	2	任务2: 钢珠 O→+5→0→-5 (停), <5s
TASK_AB_STAB	3	任务3: A→B + 钢珠稳定在O, ≤8s
TASK_LAP_STAB	4	任务4: 一圈回A + 钢珠稳定在O, ≤30s
TASK_LAP_POS	5	任务5: 一圈回A + 钢珠指定位置, ≤30s

子状态 (Sub_State):

宏	值	说明
SUB_IDLE	0	空闲

宏	值	说明
SUB_INIT	1	初始化 (设置速度, 开始计时)
SUB_RUNNING	2	运行中
SUB_DONE	3	完成 (停车但保持控球, 超时也会进入此状态)

7.2 任务详细说明

任务1 (纯循迹):

- 只巡线, 不控球
- 速度: TASK1_SPEED = 316 mm/s
- 经过 4 个途经点 (A/B/C/D) 后停在 A 点
- 超时 20s 自动停车

任务2 (钢珠控制):

- 小车不动, 仅舵机控制
- 流程: O → +5cm (停稳) → 0cm (停稳) → -5cm (停稳)
- 每个阶段等待球接近目标 (≤0.5cm) 连续 30 帧才切换
- 超时 5s
- 任务完成后持续保持球在 -5cm 位置

任务3 (A→B + 控球):

- 巡线 + 舵机控球同时进行
- 速度: TASK3_SPEED = 130 mm/s
- 从 A 点出发, 经过 1 个途经点到达 B 点
- 行驶中球稳定在中心 O
- 超时 8s 停车, 停车后保持控球

任务4 (一圈 + 控球):

- 巡线 + 舵机控球同时进行
- 速度: TASK4_SPEED = 222 mm/s
- 顺时针一圈, 经过 4 个途经点回到 A
- 行驶中球稳定在中心 O
- 超时 30s 停车, 停车后保持控球

任务5 (指定位置 + 控球):

- 同任务4, 但球的目标位置可指定 (Target_Position)
- 速度: TASK5_SPEED = 222 mm/s
- 超时 30s 停车, 停车后保持控球

八、关键代码讲解

8.1 K230 控球 — K230_Servo_Control() (app_k230.c)

```
void K230_Servo_Control(float target_cm)
{
    // 解析 K230 串口帧 (UART3 中断自动收数据)
    int r; do { r = parse_one(g_buf, g_wr); } while (r > 0);
    if (!g_valid) return;

    // 坐标转换 cm -> 像素 (双向 px/cm 补偿)
    float px_per_cm;
    if (target_cm >= 0.0f)
        px_per_cm = (K230_MAX - K230_CENTER) / PIPE_HALF_CM;  // 右侧
    else
        px_per_cm = (K230_CENTER - K230_MIN) / PIPE_HALF_CM;  // 左侧
    float target_px = K230_CENTER + target_cm * px_per_cm;

    // 死区 + 位置式 PID + 速率限制 + PWM 输出
    float error = target_px - Kalman_Filter(&kf_x, g_x);
    if (error > -DEADBAND_PX && error < DEADBAND_PX) return;
    float cc = PID_To_CC_Pos(&g_pid, x_flt);
    // ... 速率限制 + CC 限幅 ...
    DL_TimerA_setCaptureCompareValue(PWM_Servo_INST, (uint16_t)cc, DL_TIMER_CC_0_INDEX);
}
```

PID 公式 (位置式, app_pid.c): `u = Kp*e + Ki*Integral(e) + Kd*(e - e_last)`

8.2 巡线 — LineWalking() (app_irtracking.c)

```
void LineWalking(void)
{
    static int8_t err = 0;
    static u8 x1~x8;
    deal_IRdata(&x1,...,&x8);        // I2C 读取八路红外

    if (x4 == 0 || x5 == 0) err = 0; // 中间黑线, 直行
    else if (x3 == 0)       err = -1; // 左偏, 右转
    else if (x6 == 0)       err =  1; // 右偏, 左转
    // ... 更多传感器组合 ...

    pid_output_IRR = (int)(PID_IR_Calc(err));
    Motion_Car_Control(g_line_speed, 0, pid_output_IRR);
}
```

8.3 巡线+控球组合 — LineWalking_WithBall() (app_irtracking.c)

```
void LineWalking_WithBall(float ball_target_cm)
{
    // 惯性补偿: 前进 + 转弯
    float compensated = ball_target_cm;
    compensated -= BALL_INERTIA_FWD;
    if (pid_output_IRR > 0)
        compensated -= BALL_INERTIA_TURN * pid_output_IRR / 5000.0f;

    // 起步缓加速: 前 RAMP_TIME_MS 毫秒速度从 0 渐变
    uint32_t et = Get_Time() - start_time;
    if (et < RAMP_TIME_MS) {
        float ramp = (float)et / RAMP_TIME_MS;
        g_line_speed = (int)(g_line_speed * ramp);
    }

    LineWalking();                      // 巡线
    K230_Receive_Process();             // 接收 K230
    K230_Servo_Control(compensated);    // 控球
}
```

8.4 任务2 钢珠往复 (questions.c)

```
// 三阶段: +5cm(1.8s) -> 0cm(1.8s) -> -5cm(1.8s) -> DONE(保持-5)
case SUB_RUNNING: {
    static int phase = 0;
    static uint32_t t0 = 0;
    if (t0 == 0) t0 = Get_Time();

    float target_cm;
    if (phase == 0) target_cm = +5.0f;
    else if (phase == 1) target_cm = 0.0f;
    else target_cm = -5.0f;

    K230_Receive_Process();
    K230_Servo_Control(target_cm);

    if (Get_Time() - t0 >= 1800) {      // 每阶段 1.8 秒
        t0 = Get_Time();
        if (phase == 0) phase = 1;
        else if (phase == 1) phase = 2;
        else { phase = 0; Sub_State = SUB_DONE; }
    }
    break;
}
case SUB_DONE:
    K230_Receive_Process();
    K230_Servo_Control(-5.0f);     // 保持球在 -5cm 位置
    break;
```

九、操作流程

1. 接线后上电, OLED 显示 "K230 Init..." 等待 4 秒
2. 进入模式选择界面, OLED 显示 "Mode:1"
3. **长按 K1** 切换模式 (1~5)
4. **短按 K1** 确认进入当前模式
5. 小车开始执行任务, OLED 显示任务名 + 计时
6. **任务运行中短按 K1:** 暂停; **长按 K1:** 退出回模式选择
7. 任务完成/超时后, 小车停车但**控球任务继续** (球保持在目标位置)
8. 长按 K1 退出回模式选择